

Unified Substrate Definition

Numerical kernel, corrections and validation

An explicit, finite numerical specialization of the unified v0.2 formalization.

The wavefront, editable operator field, Klein carrier, binary routes, four-stage evolution and inverse/history channels remain one connected recurrence.

What this edition adds

Corrected numerical arithmetic and input validation, explicit executable ordering, honest topology and expressivity limits, and recorded validation on the available hardware.

Tom Klootwijk

Dawnwood Interactive

16 September 2026

Source lineage: original discussion -> Unified v0.2 -> numerical kernel v0.3 -> Unified / kernel v0.4. Source PDFs are preserved unchanged.

Unified Substrate Definition, version 0.4

Author attribution: Tom Klootwijk, Dawnwood Interactive. **Edition:** 16 September 2026. **Numerical profile:** DWI-N1-0.4, C++ and Vulkan compute.

This edition develops the complete unified definition in *Dawnwood_Interactive_Unified_v0.2.pdf* and retains the connected wavefront, situated operators, non-orientable carrier, branching, history and feedback. The v0.3 kernel supplied numerical bindings; v0.4 corrects numerical and runtime defects and states their mathematical limits explicitly.

References marked **Source** identify pages of the unchanged 24-page *source/double-slit-theory.pdf*. **Baseline** identifies the latest 18-page unified v0.2 formalization. Source relationships, numerical choices and established results are different categories. This document specifies the first two. The accompanying validation appendix records only work actually performed.

1. The unified Dawnwood definition

Dawnwood is a self-referential spatial computing proposal: the acting operator field belongs to the state that returns as the next input. The double pinion, Hadamard hinge, log-polar coordinates, geometric LUT, implicit binary routes, one-bit controls and RGBA roles participate in that circulation. Optional output is downstream. [Source pp. 3-16; Baseline p. 3.]

For N wavefront records and M operator records, write the whole state as $S_n=(K,W_n,L_n,C_n)$. K is the fixed carrier in this numerical profile; W contains wavefront and kinematic variables; L contains mutable situated definitions; C identifies the channel/history/inverse roles already stored with W. It is not another independent copy of memory.

The executable recurrence is:

$$\begin{aligned} L_{n+1} &= J(L_n, W_n; \text{config}, n) \\ W_{n+1} &= E(W_n, L_{n+1}; \text{config}) \\ S_{n+1} &= (K, W_{n+1}, L_{n+1}, C_{n+1}) \end{aligned}$$

One logical PSI interval is one mutation pass followed by one evolution pass. The source's designation *Universal Spatial State Automaton* remains its proposed name; it is not a universality theorem or a physical quantum-device classification.

2. Source progression and notation

The source introduces double pinion and log-polar encoding on pp. 3-4; parity on pp. 4-5; the literal FT vector on p. 6; fourth-slot Y-up on p. 7; geometry, colon, phase divergence and PSI on pp. 8-9; RGBA and inverse T on pp. 9-12; the common Klein field and operator change on pp. 12-16; GPU organization and hypothetical capacities on pp. 18-23. [Baseline p. 4.]

User proposals establish the requested architecture. Earlier generated responses add stronger assertions about physics, noise removal, memory, optimization and universality. They supply neither experimental observations nor proofs.

Here n is a uint32 logical epoch; d is route depth; sigma is a transported orientation bit; (u,v) is a chart representative; rho is logarithmic radius; theta is its phase/angle variable. T_shape is a primitive, while T_transform is a matrix. The golden ratio is used only to define the initialization angle. PSI is not a measured wall-clock or physical duration.

In equations below, $B_i(x)$ evaluates operator i's scalar body, $F_i(s)$ evaluates its situated field, and $A_i(s,x)=B_i(x+0.01 F_i(s))$. These definitions refer to the current LUT unless an old LUT is stated explicitly.

3. Double pinion, mitosis and the one-bit split

The source wavefront [0,2,0,1] is bound to two complex amplitudes $a=0+2i$ and $b=0+i$. Their squared norm is five; initialization does not normalize it to one. [Source pp. 3-6; Baseline p. 5.]

With g the final situated geometry response, the hinge/phase action is:

```
H = (1/sqrt(2)) [[1,1],[1,-1]]
p = phase_1 + 0.01 A_1(s',g) + epsilon*(2j-1)
  + 0.02 A_17(s',g) + 0.01 A_15(s',energy_old)
  + 0.01 A_24(s',g)
(a',b') = diag(exp(i*p),exp(-i*p)) H (a,b)
```

This preserves the existing norm in exact arithmetic. FP32 roundoff remains measurable; there is no corrective normalization. The pinion also provides chart transport parameters du and dv , a distinct action coupled through the same records.

Population parity is $\text{popcount}(x) \bmod 2$; integer even/odd is $x \& 1$. Both give (1,0) for (7,54), but differ for 3. Routing consumes actual integer controls. Neither parity nor a classical complex-number calculation constitutes a physical measurement.

4. Log-polar encoding and the FT wavefront

The numerical binding is $\rho = \ln(r)$, $r = \exp(\rho) > 0$. Locally, $q = (r \cos(t), r \sin(t))$, where $t = \theta + 0.01 A_2(s, \rho)$. Multiplying radius by $k > 0$ adds $\ln(k)$ to ρ . No quantized radius table or arbitrary information-capacity increase follows from this identity. [Source pp. 3-4, 6, 11-12; Baseline p. 6.]

The assignment $756 \rightarrow [0,2,0,1]$ is retained as a literal source instance. The source supplies no general timestamp encoder. Independently interpreting those four numbers as samples in an unnormalized forward DFT yields $[3, -i, -3, i]$; this interpretation is not substituted for the supplied initial amplitudes.

Coordinates and operator fields use FP32. No opacity clamp is applied to ρ . Exponential overflow, underflow, accumulated angle error and finite bytecode arithmetic remain possible outside a suitable operating range; finite input values alone do not prove bounded future trajectories.

5. PSI, jitter and the fourth RK4 slot

The integer generator is `xorshift32`: sequentially XOR x with $x \ll 13$, $x \gg 17$ and $x \ll 5$, using uint32 arithmetic. For a wavefront, $r_{\text{next}} = \text{RNG}(r_{\text{old}} \text{ XOR seed}_{20} \text{ XOR program}_{20})$, and j is its low bit when jitter is enabled, otherwise zero. This is deterministic control, not a specified physical random process. [Source pp. 7-9, 14-16; Baseline p. 7.]

Define $h = dt * (1 + 0.01 A_{29}(s, 0))$ and $\delta = A_{13}(s, \theta - 2 \theta_{\text{previous}} + \theta_{\text{previous_previous}})$. δ is an unscaled second difference over logical epochs, not a time-normalized second derivative. The authored PSI body modulates h ; positive base dt alone does not guarantee positive h for every edited body.

For $x = (u, v, \rho, \theta)$, with other state fields held during stages:

```
k1 = f(x)
k2 = f(x + h*k1/2)
k3 = f(x + h*k2/2)
y = yup * A_16(s, 1)
k4 = f(x + h*k3 + (0, 2*y, 0, 0))
x' = x + h*(k1 + 2*k2 + 2*k3 + k4)/6
```

The implementation uses a non-unrolled four-iteration loop with one derivative call site and stores all four slopes. It performs two separate additions of y to the fourth stage. This explicitly binds the source's fourth-slot event to a

position before derivative evaluation. Baseline equation 13 left the action symbolic. Ordinary RK4 convergence order and alleged noise cancellation are not established for this modified rule.

With jitter disabled, $j=0$ still gives a negative epsilon offset and negative mutation sign. This is a fixed-bit ablation. With feedback disabled, operator mutation freezes; wavefront/history recurrence continues.

6. Geometric LUT, double dot and phyllotaxis

The primitive group remains connected to the derivative: T is the minimum of two box fields; pyramid is a triangular section extruded through $|z| \leq 0.06$; circle is $\sqrt{xx+yy}$ -radius; cone uses capped-cone side/base segment distances with an inside sign; sphere is Euclidean radius minus operator radius; apex is distance to the origin. [Source pp. 8-9; Baseline p. 8.]

The apex is unsigned. A union, blend or transformed scalar field need not remain an exact metric signed distance. For each primitive, F_i applies the current scalar body to its local field. Geometry is $g=A_0(s, (F_7+\dots+F_{12})/6)$, using explicit primitive kinds 0-5.

Let $P=[[du,dv],[dv,du]]$ from pinion 5 and $Q=[[qx,qy],[qy,qx]]$ from the decoded polar vector. The colon binding is $c=A_6(s,Q:P)$, where $Q:P=2(qxdu+qydv)$. Set $b=A_{14}(s,g+\text{coupling}_6*c+0.01 A_4(s,\delta))$. For selected record t , $z=A_t(s,b)$, except downstream markers 25/26 use $z=b$. The derivative is:

```
u_dot = du_5 + 0.001 A_5(s,response)
        + 0.02 sin(theta+2*pi*v) + fieldScale*sin(z)
v_dot = dv_5 + 0.02 cos(2*pi*v+theta)
        + fieldScale*sin(z+phase_2)
rho_dot = radialRate*(sin(theta+phase_t)-0.1*rho)
          + 0.001*response
theta_dot = phaseRate + 0.02 A_3(s,z) + 0.01*delta
```

For downstream markers, phase_t in ρ_{dot} is zero. The live phyllotaxis body affects b . Initialization separately uses golden angle $\gamma=2.39996322972865332$ and square-root radial progression. This does not establish uniform Klein surface-area sampling or optimal GPU locality.

7. The Klein bottle as the common operator field

The numerical carrier is the quotient $(u+1,v) \sim (u,-v)$, $(u,v+1) \sim (u,v)$. With $m=\text{floor}(u)$, canonicalization maps u to $u-m$, v to $\text{frac}((-1)^m v)$, and σ to $\sigma \text{ XOR parity}(m)$. [Source pp. 12-16; Baseline p. 9.]

For inspection, $U=2\pi u$ and $V=2\pi v$ define the four-dimensional embedding:

$$E(u,v) = ((2+\cos(V))\cos(U), (2+\cos(V))\sin(U), \sin(V)\cos(U/2), \sin(V)\sin(U/2))$$

It obeys $E(u+1,v)=E(u,-v)$. This is not a globally signed solid boundary in ordinary three-dimensional space.

Each operator supplies an anchor (a,b) . A field query first canonicalizes a local copy of the wavefront. It minimizes $xx+yy$ over $m=-1,0,1$, where $x=u-a-m$ and $y=v-(-1)^m b$ rounded to its nearest vertical translate. *Strict comparisons retain the first horizontal minimizer; a vertical +0.5 tie maps to -0.5. The primitive receives $x+0.01r\cos(\theta)$, $y+0.01r\sin(\theta)$, and $z=(-1)^\sigma 0.2*(r-0.35)$.*

In v0.4, floor-based floating parity avoids undefined large float-to-uint seam conversions. A remainder rounded to exactly one is represented by the greatest FP32 value below one. Field queries canonicalize transient RK stages before the three-image search; they do not rewrite stored stage slopes.

This profile defines a canonical-chart discrete recurrence. Smooth quotient-compatible flow would additionally require $f(u+1,-v)=\text{diag}(1,-1)f(u,v)$, and consistent operator-frame transport for asymmetric primitives. The present

derivative and fields do not establish those properties. Correct position wrapping must not be advertised as a proof of smooth intrinsic dynamics. The carrier is fixed; its situated scalar operators evolve.

8. One-bit tree and zero-pointer routing

Routing uses $\text{child}(i,b)=2*i+1+b$, starting at zero. It is an implicit binary decision tree; the source's term BST does not supply a search-key comparator or ordering invariant. [Source pp. 8-9, 13-15, 22; Baseline p. 10.]

At level l , the branch XORs bit l of the old RNG word, j , wavefront orientation, the current node's low flag, population parity of seed_28 XOR program_28, the low bit of flags_27 XOR program_27, and the low bit of seed_21 XOR program_21. There is no modulo aliasing of a missing operator address. The complete route requires $2^{(d+1)}-1 \leq M$ and $d \leq 30$.

The catalogue order is an implementation choice retained from the baseline:

Index	Entry	Numerical role
0	klein	Geometry aggregate
1	hadamard	Hinge phase response
2	phi	Polar phase encoding
3	rk4	Angular derivative response
4	phyllotaxis	Divergence/blend response
5	pinion	Transport and coupling
6	double_dot	Colon response
7	T_shape	Two-box primitive
8	pyramid	Triangular extrusion
9	circle	Radial section
10	cone	Capped cone
11	sphere	Spherical field
12	apex	Point-distance field
13	delta_phi	Phase difference response
14	blend	Coupled scalar response
15	wavefront	Previous-energy phase input
16	y_up	Fourth-stage displacement
17	crystal	Output phase input
18	inverse_T	Inverse response feedback
19	T_transform	Rotation/shear parameters
20	jitter	RNG integer context
21	bst	Branch integer context
22	return	Final horizontal increment
23	phase_history	History response
24	dichromatic	Paired phase input
25	bayer	Downstream threshold readout
26	bc5	Downstream packing experiment
27	split	Branch integer context
28	parity	Population-parity context
29	psi	Local step modulation
30	mutation	Feedback mutation rate

9. Dichromatic RGBA and A as inverse T

R and G designate the complex pinion streams, requiring four FP32 amplitude components together. B designates finite history. A is an integer reference to the inverse matrix stored in the same wavefront record, not scalar opacity. [Source pp. 9-12; Baseline p. 11.]

Set $\text{angle} = \text{theta}' + \text{phase_19} + 0.01 A_19(s', g)$, $c = \cos(\text{angle})$, $s = \sin(\text{angle})$, and $q = \text{shear_19}$. Then:

```
T = [[c, c*q-s], [s, s*q+c]]
D = c*c+s*s
T_inverse = [[s*q+c, -(c*q-s)], [-s, c]] / D
response' = A_18(s', inverse00*g + inverse01*delta)
history' = w*history_old + (1-w)*A_23(s',
    k1.v+k2.v+k3.v+k4.v+j+response')
```

The stable determinant D exploits rotation-times-shear structure. It avoids cancellation of large shear terms present in the expanded determinant. Large shear can still produce an ill-conditioned inverse; this correction is not a conditioning guarantee. The response feeds subsequent radial motion and mutation. Inversion does not inherently erase independently added channel noise.

10. The complete PSI circulation

The numerical schedule specializes the baseline's written symbolic order. [Baseline p. 12; Source pp. 13-17.]

1. Read old state and old LUT; update every operator from reproducible feedback samples.
2. Establish a compute-write to compute-read dependency.
3. Read old wavefront and new LUT; determine RNG bit and route; compute h and phase difference.
4. Evaluate four derivative stages, with double Y-up only in the fourth.
5. Add $h * 0.001 A_22(s, \text{field_old})$ to u; canonicalize the resulting position.
6. Recompute geometry; update complex amplitudes, inverse matrix, response and history.
7. Store energy, four vertical slopes, route, reference, RNG, prior phases, jitter and embedding.
8. Establish the next dependency, exchange old/new buffers and increment the epoch.

Thus the implemented amplitude hinge follows kinematic integration. The baseline's named actions do not establish that these numerical operations commute. This order is explicit and versioned.

Bayer reads completed state using ordered-threshold dithering. The source's optional name is ambiguous with a camera color-filter mosaic; this binding does not simulate a sensor. BC5 is a separate two-channel block-format experiment. Neither downstream marker injects compressed pixels into feedback.

11. Self-reference, operator change and expression

An operator record contains position, geometric parameters, phase, gain, coupling, shear, transport, mutation rate, program, seed, kind and flags. Its scalar program has eight four-bit instructions, low nibble first: identity, add gain, subtract gain, multiply coupling, sine, cosine, absolute value, negate, square. The default 0x341 means add, sine, multiply, then identities. Unknown instructions are invalid inputs. [Baseline p. 13.]

For operator i, feedback samples are $a = (17i + n) \bmod N$ and $b = (31i + n + 1) \bmod N$. v0.4 evaluates these mathematical indices through overflow-safe reduced uint32 additions. Old record 30 changes all records, including itself; old record 5 supplies transport.

With old states A and B and advanced operator seed, define j_i from seed XOR A.rng XOR B.route. If jitter is disabled, $j_i=0$. Define $\text{rate}=\text{mutationRate}_{30}(1+0.1 A_{30}(A,A.\text{field}))$. The update is:

```
u += dt*(du_5 + rate*sin(A.theta) + 0.001 F_5(A))
v += dt*(dv_5 + rate*cos(B.theta)); canonicalize
phase += rate*(2*j_i-1) + 0.001*dt*(A.history+B.response)
gain += 0.1*rate*sin(F_i(A)+A.theta)
shear += 0.1*rate*cos(B.field)
```

The gain field query uses the operator's newly moved position. When $j_i=1$, its first nibble toggles add/subtract or sine/cosine; other opcodes remain unchanged. Changed numerical bodies affect future action without native shader recompilation.

This is finite selfmodification. Unlike the baseline's arbitrary symbolic JSON expressions, this instruction set cannot express an arbitrary new native algorithm. Neural, database and universal-computation mappings remain design vocabulary requiring concrete encodings and evidence.

12. GPU substrate and resident organization

The active targets are the owner's GTX 1650 Ti laptop with 4 GB and POCO X7 Pro 12 GB system-RAM edition, superseding the source's RTX/12 GB scenario. Actual device identity, heap sizes, feature support and execution must be reported separately. Phone system RAM is not dedicated VRAM. [Source pp. 18-19; Baseline p. 14.]

Vulkan mutation assigns one invocation per operator; evolution assigns one per state. Each output has one writer. Old/new buffers and dependencies provide the declared ordering; no cross-workgroup spin barrier is required. The workgroup size is 64. The numerical interface uses FP32, uint32 and storage buffers, with device limits checked before allocation. Version 0.4 checks padded allocation requirements against the specific memory heaps used, preserves upload-to-readback visibility even with zero evolution epochs, and treats reported Vulkan validation errors as run failures.

The shader build marks real loops `DontUnroll`, avoids artificial loop constructs in return helpers, and uses offline optimization. Sharing one derivative call site across four stages reduces repeated compiler expansion. These hints do not change the recurrence or guarantee a particular driver memory footprint.

Reuse avoids a per-interval host upload of changed operators. It does not guarantee permanent on-chip cache residence, zero bus traffic or optimal scheduling. Initialization and requested readback transfer data. Compilation, a software Vulkan run and an APK each establish different things from execution on either physical target.

13. Packing scenarios and operation-level cost

A wavefront is 128 bytes: 28 FP32 values and four uint32 values. An operator is 64 bytes: 12 FP32 values and four uint32 values. Config is 64 bytes. Two state buffers and two LUT buffers require $256N+128M$ bytes; the reusable transfer buffer requires $\max(128N, 64M)$ more. Padding and other allocations are additional. Heap placement determines which budgets they consume. [Source pp. 20-22; Baseline p. 15.]

BC5 uses 16 bytes per two-channel 4x4 block, averaging one byte per texel; reconstruction is generally approximate. It cannot silently replace the exact 128-byte state. The source's approximately 12.88 billion one-byte elements correspond to a hypothetical 12 GiB pool, not twelve billion complete wavefront records. Its 24-48 billion operational-state scenario gives no full-state byte width.

For fixed instruction width and primitive count, an epoch performs $\Theta(M+N*d)$ work. With default fixed d , it is linear in N . A route costs d child operations. One lookup or shared-control change is a different workload from advancing all states. Removing two actually present uint32 pointers saves eight bytes per record; this layout already has no child pointers.

14. Editable bindings and defaults

The following values define the default profile, rather than equations attributed to the source. [Baseline p. 16.]

Configuration	Default
N, M, epoch, depth	256, 31, 0, 4
dt, yup, epsilon	0.01, 0.0005, 0.125
mutation, jitter, feedback	1, 1, 1
phaseRate, radialRate	0.125, 0.01
historyWeight, fieldScale	0.75, 0.005
Reserved configuration words	Zero

For state i , let $r_i = \sqrt{(i+0.5)/N}$ and $t_i = i\gamma$. Initialize $(u,v) = (0.5+0.45r_i \cos(t_i), 0.5+0.45r_i \sin(t_i))$, $\rho = \ln(0.2+0.4r_i)$, and $\theta = t_i \bmod 2\pi$. Prior phase samples equal θ ; inverse is identity; amplitudes and energy are the source fixture. RNG is $\text{RNG}(756 \text{ XOR } (i747796405+2891336453))$, replacing a zero result with 756. Other state values start at zero except $\alpha_{\text{ref}}=i$ and the calculated embedding.

Operator i uses $r_i = \sqrt{(i+0.5)/M}$, position $0.5+0.4r_i(\cos(t_i), \sin(t_i))$, radius $= 0.18+0.005(i \bmod 5)$, height $= 0.25$, phase $= 0.02i$, gain $= 0.1$, coupling $= 0.9$, shear $= 0.1 \sin(t_i)$, $(du, dv) = 0.03(\cos(t_i), \sin(t_i))$, mutationRate $= 0.0002$, program $= 0x341$, kind $= i$ and zero flags/reserved. Seed is $\text{RNG}(756+17*i)$, replacing zero with one.

Shared equations live in `include/numeric_types.inc` and `include/numeric_evolve.inc`. Changing them requires regenerated shader artifacts. CLI program edits and checkpoint record edits change supported live definitions. Authoring remains bounded by opcode, representation, finite-value and device constraints; v0.2's unbounded symbolic editing is not promised here.

15. Reproduction, checkpoints and validation boundaries

The CPU and Vulkan implementations share numerical equations. Their agreement checks implementation, layout and scheduling; it does not independently derive those equations or verify physical claims. Meaningful evidence distinguishes analytic fixtures, backend comparison, checkpoint continuation, operator sensitivity, real-device measurement and external experiment. [Baseline p. 17.]

Representative commands, using the executable path appropriate to the build, are:

```
dawnwood selftest
dawnwood verify --device "1650 Ti" --count 256 --steps 16
dawnwood run --backend cpu --count 256 --steps 16
dawnwood run --resume state.dwk --steps 16 --checkpoint continued.dwk
```

These are reproduction instructions, not claims that these runs succeeded. Per-epoch comparisons should retain the first divergent field and epoch. Backend tolerance must not be relaxed merely to hide a failure. Operator sensitivity must inspect numerical state, not only the edited program word. Norm, chart health and inverse residual each address a different invariant. A run health pass denotes finite and structurally valid records; it does not impose the separate norm-error fixture threshold.

Checkpoint layout remains `DWKN003`: eight-byte magic, two record-size words, 64-byte Config, N state records and M operator records. Thus the 80-byte header and existing record ABI remain compatible. Valid v0.3 checkpoints can supply initial state to corrected v0.4 equations; this does not promise bitwise replay of the old trajectory. Unknown programs or invalid records must be rejected rather than interpreted as successful no-ops.

All shared floating constants now explicitly use FP32 literals. In v0.3, unsuffixed C++ constants promoted some intermediates to double while GLSL evaluated them in FP32; v0.4 removes that discrepancy and configures the Windows CPU build with `/fp:strict`. GLSL `precise` qualifiers, including precise return temporaries inside called functions, generate SPIR-V `NoContraction` decorations for the basic arithmetic. This matches the CPU no-contraction policy. Transcendental implementations, rounding and accumulated numerical error can still differ by backend. Agreement is tolerance-based, not a promise of bitwise backend identity.

Runtime hardening includes complete checkpoint/snapshot validation, strict CLI floating-value parsing, rejection of a vacuous zero-epoch verification, and full configuration reporting at adequate precision. Invalid input must fail before entering the numerical recurrence. Epoch exhaustion is an explicit limit. Detailed actual outcomes, unavailable hardware and unresolved failures belong in the validation appendix and raw logs, not inferred from file presence.

Source and edition record

The unchanged 24-page source supplies the conceptual relationships. Unified v0.2 supplies this edition's complete fifteen-section baseline. Kernel v0.3 supplies DWI-N1's executable choices. Unified v0.4 incorporates the reviewed corrections: matching literal precision, mathematical feedback indexing without overflow, safer seam parity/reminders, canonical transient field queries and structured inverse determinant, together with runtime/input-validation improvements documented in the release record.

This is a new numerical revision with a compatible record layout. Smooth transported-frame dynamics, general universality, physical double-slit advances, automatic error correction, permanent cache residence and hypothetical capacity/performance advantages remain unproved. Preserving those distinctions keeps the source's self-referential architecture intact while making its executable meaning reviewable.

Tom Klootwijk - Dawnwood Interactive - Unified Substrate Definition v0.4.

Appendix A. Validation record

Date: 16 September 2026. **Partially validated; two numerical agreement failures remain.** The final kernels build and execute on both physical targets. No test sources were added. Existing fixtures, existing comparison tools and recorded manual invocations were used. The original v0.3 archive and the latest supplied v0.2 formalization remain unchanged.

Results on the final implementation

Check	Recorded result
Desktop C++ and both SPIR-V shaders	Build and SPIR-V validation pass
Existing desktop CPU fixtures	30/30 pass
Existing Python source/packing checks	17/17 pass
Active operator numerical sensitivity	29/29 pass; Bayer/BC5 are downstream
CPU checkpoint replay	Exact digest match
GTX 1650 Ti, 128 states x 16 epochs	All-word CPU/GPU comparison passes
GTX 1650 Ti, 257 states x 128 requested epochs	Fails at epoch 96; 95 preceding epochs pass
Desktop core and synchronization validation	Zero errors and zero warnings in the final long comparison
Near uint32 epoch limit, 257 states	One-step comparison passes through epoch 4294967295; overflow request rejected
Zero-epoch GPU readback	Passes with validation; ordinary zero-step runs remain supported
Invalid opcode checkpoint, malformed float, zero-step verify	Each rejected with nonzero exit
Android arm64 library and APK	Build successfully with recorded alternative toolchain
Actual POCO X7 Pro CPU fixtures	30/30 pass
Actual POCO X7 Pro GPU pipeline creation	Passes after sharing one RK derivative call site
POCO, 128 states x 16 requested epochs	Comparison fails at epoch 1
POCO, 4096 states x 64 epochs	Completes; finite valid records, no out-of-chart states

The final desktop suite is `claims_rk_loop/`; final phone reports are `phone_rk_loop/. *.command.json`, `stdout` and `stderr` retain actual commands, exit codes and timings. Earlier directories record earlier revisions and retained failures; they do not override the final table. `summary.json` is the machine-readable edition summary.

Numerical disagreement is retained

All comparisons use the original rule $\text{abs}(\text{cpu-gpu}) \leq 1\text{e-}5 + 2\text{e-}5 \cdot \max(\text{abs}(\text{cpu}), \text{abs}(\text{gpu}))$; integer words must be exact. The tolerance was not relaxed.

The GTX's first final mismatch is **epoch 96, state[222].ai**, CPU **-0.0367288142**, Vulkan **-0.0367168337**. There is one out-of-tolerance float at that epoch, with no integer mismatch, nonfinite value or invalid record. Across the checked epochs, maximum absolute error is **4.768371582e-5** and maximum scaled error is **1.116069538**.

Starting both backends from the identical saved CPU epoch-95 state and advancing once passes, with maximum absolute error **1.907348633e-6**. This supports accumulated backend rounding drift as a diagnosis; it does not isolate every elementary-function discrepancy or establish an unlimited horizon.

The POCO's first mismatch is **epoch 1, state[1].ar**, CPU **-0.289921224**, Vulkan **-0.289942235**. There are **135** out-of-tolerance floats, **zero** integer mismatches, and no nonfinite or invalid records. Maximum absolute error is **4.159659147e-5**; maximum scaled error is **3.415186812**. The phone's comparison therefore does not pass even though it now executes the kernel. Residual backend arithmetic must be investigated before claiming the same numerical accuracy on Mali. No custom transcendental approximation was silently substituted.

The independent 4096-state phone run completed all 64 epochs with maximum reported energy error from the source norm five of **0.000226974**. A health pass checks finite/structural validity; it is not a backend-equivalence certificate. Android Vulkan validation layers were not installed for this run, so zero recorded layer errors on the phone is not a validation-layer result.

The phone compilation improvement

Initial real-device attempts failed creating the evolve pipeline, including with a one-state probe. Own-process logs showed Scudo allocator exhaustion and native crashes; approximately 2.28 GB process RSS was observed during a compiler attempt. These were driver/compiler allocations, not the live-state buffer payload.

Explicit loop controls, removal of artificial return-macro loops and offline optimization alone did not resolve the failure. Replacing four expanded RK derivative call sites with one call in a non-unrolled four-iteration loop reduced optimized evolution SPIR-V from **578,760 to 268,608 bytes** and instructions from **36,200 to 16,724**. All four slope records and two separate fourth-stage Y-up additions remain. The 257-state, 95-epoch CPU state/LUT digest stayed **0380ebf43a2cfb07**, and the existing Y-up fixture passed. With this change the actual Mali pipeline compiles and the 64-epoch run completes. This demonstrates a compilation-resource improvement; it is not a claim of universal driver compatibility.

phone/, phone_loop_control/ and phone_optimized/ preserve unsuccessful attempts. Early broad system logcat files are excluded from the release archive; the retained release evidence uses the application's own PID logs. Final build uses `tools/build_shaders.py --optimize` and retains arithmetic NoContraction and loop DontUnroll decorations.

Hardware, memory and build provenance

Desktop: **NVIDIA GeForce GTX 1650 Ti with Max-Q Design**, driver **581.80**, NVML-reported **4096 MiB**. Windows 11 build 26200; MSVC 19.44; CMake 3.31.6; shader tools from NDK 29.0.14206865. Vulkan headers were copied separately from the installed NDK; the import library was generated from the installed Vulkan loader's exported functions. The first baseline configuration accidentally included the whole Android sysroot; that failed build and the corrected configuration remain logged. It was a local setup error, not a kernel defect.

The official LunarG 1.4.357.0 validation layer was extracted into a local directory and its download checksum verified. No system SDK installation/registry changes were needed. Final desktop commands set `DAWNWOOD_VALIDATION=1`, `VK_LAYER_PATH` and `VK_LAYER_VALIDATE_SYNC=1` per process. The first layer run emitted a deprecated-setting warning; later recorded runs use the new setting and report zero warnings. See `layers_provenance.json`.

Phone: **POCO X7 Pro**, model **2412DPC0AG**, Android API **36**, arm64-v8a, **Mali-G720 MC7**, raw driver version **205524992**. The device reports **11,861,921,792 bytes of system RAM**, not dedicated VRAM. The final 4096-state run allocates **1,576,832 padded buffer bytes**, uploads **526,272 bytes** and downloads **526,272 bytes**. These omit driver/pipeline allocations. The phone does not expose `VK_EXT_memory_budget`; heap size is a fallback limit, not free RAM. BC5 sampled-image and BC compression support are **false**; EAC/ASTC sampled support is true. The exact recurrent state remains in storage buffers.

The APK uses pinned Gradle 8.9 / AGP 8.7.3 / SDK 35 with the available **NDK 29.0.14206865** and **CMake 3.31.6**, selected by `android-local-toolchain.gradle`. Original NDK/CMake pins are unchanged. The authorized phone ran the built APK; it was force-stopped after measurement. Initial failures and the final successful execution are distinct from the remaining failed numerical comparison.

Timing observations

Three desktop samples per count, 16 epochs, batch size eight; medians below. These are measurements for the final build, not a controlled before/after speedup study.

States	CPU host compute ms	GTX device-timestamp ms
256	45.967	3.336
1024	219.892	3.414
4096	734.800	4.146

CPU and GPU use different timing scopes; GPU initialization, pipeline compilation and transfers are excluded from device timestamps. The final phone's single 4096-state, 64-epoch run reports **98.019 ms** from device timestamps, with Android thermal status zero. No sustained thermal or cross-device speedup claim follows.

Scope and reproducibility

Final equations are in the shared include files; the full definition is `docs/FORMALIZATION_v0.4.md`. Reproduce desktop source-linked checks with `python tools/run_claims.py --binary build/Release/dawnwood.exe --device "1650 Ti" --bench --out results/reproduction`. Reproduce the known long failure with `dawnwood verify --device "1650 Ti" --count 257 --steps 128`. Use the recorded Android init script and `tools/test_phone.py` for the phone; its comparison correctly returns a failing overall status.

Record layout remains `DWKN0003`; valid old snapshots seed corrected v0.4 dynamics, not bitwise v0.3 trajectories. The synthetic near-limit checkpoint sets only the epoch of a zero-step snapshot to 4294967294; it is not a claim of running billions of intervals. No smooth Klein tangent-flow proof, physical detector experiment, universality proof, automatic noise deletion, permanent cache residence or lossless full-state BC5 representation was supplied by these checks.

Appendix B. Sources and edition identity

The user-confirmed baseline is the unchanged 18-page *Dawnwood_Interactive_Unified_v0.2.pdf*. Its SHA-256 is recorded in `source/formalization_lineage_v0.4.json` alongside the unchanged 24-page source discussion. `docs/CHANGES_v0.4.md` maps this edition's concrete corrections. Shared implementation files and the recorded commands are the authority for numerical details and observed outcomes.

Primary API references: [Khronos GLSL specification](#), [Khronos synchronization examples](#), and [official LunarG SDK distribution](#). API documentation defines the implementation contract; it does not substantiate the source's physical claims.